

Libro - Library Management System

Libro is a robust and efficient Library Management System built with Node.js, Express, and MySQL. It provides a comprehensive set of APIs to manage books, borrowers, and the borrowing process.

Features

- **Book Management:** Add, update, delete, and list books.
- **Borrower Management:** Manage library members, including their profiles and registration.
- **Borrowing System:** Track borrowed books, manage return dates, and identify overdue items.
- **Search Functionality:** Search for books by title, author, or ISBN.

Prerequisites

Before you begin, ensure you have the following installed:

- [Node.js](#) (v14 or higher recommended)
- [MySQL Server](#)

Getting Started

1. Clone the Repository

```
git clone <repository-url>  
cd LIBRO
```

2. Install Dependencies

```
npm install
```

3. Database Configuration

1. Create a MySQL database named `libro`.

2. Update the database credentials in [DataBaseConf.js](#):

```
const mysql = require('mysql2');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'your_username',
  password: 'your_password',
  database: 'libro' in my Case I named it books_db
});
```

3. Run the SQL script located in [SCHEMA_QUERY.SQL](#) to set up the required tables.

4. Start the Server

```
node server.js
```

The server will start running on <http://localhost:3000>.

API Documentation

Books

Get All Books

- **Endpoint:** [GET /books](#)
- **Description:** Retrieves a list of all books in the library.
- **Output:**
 - 200 OK: [{ "id": 1, "title": "Pharaoh book", "author": "Mohamed" }, ...]

Search Books

- **Endpoint:** [GET /books/search](#)
- **Query Parameter:** [q](#) (Search term for title, author, or ISBN)
- **Description:** Searches for books matching the search term.
- **Output:**
 - 200 OK: [{ "id": 1, "title": "...", "author": "...", "isbn": "...", "quantity": 5, "shelf_location": "...", ... }]

Add a Book

- Endpoint: POST /books
- Input (JSON Body):
- Output:
 - 201 Created: { "message": "Great! The book has been successfully added to the library." }
 - 400 Bad Request: { "message": "Title and Author are required" }
 - 500 Internal Server Error: { "message": "We could not add the book. Please check if the ISBN is unique and try again." }

Update a Book

- Endpoint: PUT /books/:id
- Input (JSON Body): Same fields as **Add a Book**.
- Output:
 - 200 OK: { "message": "The book information has been updated successfully." }
 - 500 Internal Server Error: { "message": "We ran into an issue updating the book details. Please try again." }

Delete a Book

- Endpoint: DELETE /books/:id
 - Output:
 - 200 OK: { "message": "The book has been successfully removed from the system." }
 - 500 Internal Server Error: { "message": "We could not remove the book. It might be linked to an active borrowing record." }
-

Borrowers

Get All Borrowers

- Endpoint: GET /borrowers
- Output:
 - 200 OK: [{ "id": 1, "name": "Mohamed", "email": "mohamed@example.com", "registered_date": "2024-03-20" }, ...]

Add a Borrower

- Endpoint: POST /borrowers
- Input (JSON Body):
- Output:
 - 201 Created: { "message": "Welcome! The borrower has been successfully registered." }
 - 400 Bad Request: { "message": "Name and Email are required" }
 - 500 Internal Server Error: { "message": "We could not register this borrower. Please ensure the email address is unique." }

Update a Borrower

- Endpoint: PUT /borrowers/:id
- Input (JSON Body): Same fields as **Add a Borrower**.
- Output:
 - 200 OK: { "message": "The borrower profile has been successfully updated." }
 - 500 Internal Server Error: { "message": "Something went wrong while updating the borrower profile." }

Delete a Borrower

- Endpoint: DELETE /borrowers/:id
 - Output:
 - 200 OK: { "message": "The borrower record has been successfully deleted." }
 - 500 Internal Server Error: { "message": "This borrower cannot be removed, possibly due to active borrowing records." }
-

Borrowing Process

Borrow a Book

- Endpoint: POST /api/borrow
- Input (JSON Body):

- **Output:**

- 200 OK: { "message": "The book has been successfully checked out." }
- 500 Internal Server Error: { "message": "The borrowing process failed. Please ensure the book and borrower IDs are correct." }

Return a Book

- **Endpoint:** PUT /api/return/:id

- **Description:** Marks a specific borrowing process record as returned (sets return_date to current date).

- **Output:**

- 200 OK: { "message": "The book has been successfully returned. Thank you!" }
- 500 Internal Server Error: { "message": "There was an issue marking the book as returned." }

Get Borrowed Books

- **Endpoint:** GET /api/borrowed/:borrower_id

- **Description:** Lists all currently borrowed books (not yet returned) for a specific borrower.

- **Output:**

- 200 OK: [{ "id": 1, "borrower_id": 1, "book_id": 2, "borrow_date": "...", "due_date": "...", "return_date": null }, ...]

Get Overdue Books

- **Endpoint:** GET /api/overdue

- **Description:** Lists all books where the due_date has passed and they haven't been returned.

- **Output:**

- 200 OK: [{ "id": 1, "borrower_id": 1, "book_id": 2, "borrow_date": "...", "due_date": "...", "return_date": null }, ...]

Export Data (CSV)

- **Export Overdue Last Month:** GET /api/export/overdue

- Downloads a CSV of all books overdue from the last month.

- **Export All Borrows Last Month:** GET /api/export/all

- Downloads a CSV of all borrowing activity from the last month.

- **Export by Date Range:** GET /api/export/range?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
 - Downloads a CSV of all borrowing activity within a specific period.
- **Get Analytical Report:** GET /api/analytics?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
 - Returns statistics (total borrows, returned, overdue) for a specific period.

Project Structure

- app.js: Express application setup and middleware.
- server.js: Server entry point.
- routes/: API route definitions.
- controllers/: Request handling logic.
- models/: Database interaction logic.
- config/: Configuration files (e.g., database).
- DB_Schema/: SQL scripts for database setup.